CS & IT ENGINEERING

'C' Programming

Pointers & Arrays

Lecture No.- 02

By- Satya sir

Recap of Previous Lecture

- Pointer
- Types of Pointers
- Declaration & Initialization

Topics to be Covered

- Types of Pointers
- Pointer datatype
- Pointer size
- Pointer Arithmetic
- 1-D arrays

Topic : Pointers

Void Pointer (Generic Pointer)

```
Void *P;
```

A Pointer which can Point to any type.

Ex:

```
int x = 5;
Void *P;
P=(int)&x;
```

NULL Pointer

```
datatype *Pointer = NULL;
```

A Pointer, which Points to NULL.

Ex:

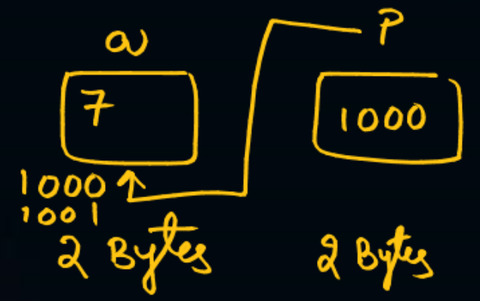
```
int *p= NULL;
float *p= NULL;
char *p= NULL;
Struct S *p= NULL;
FILE *p= NULL;
```

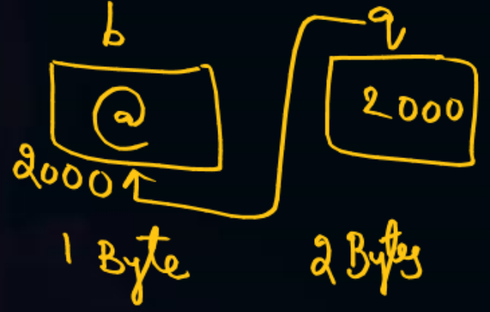
Topic : Pointers

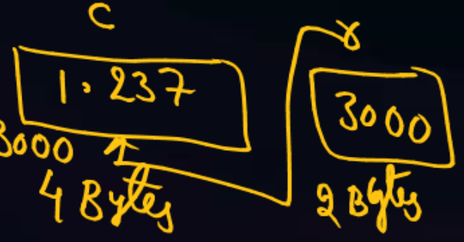
Pointer default datatype : unsigned (+ve only) Integer (whole Number)

Let 16-bit Processor

```
int   a=7, *P=&a;
char  b='@', *q=&b;
float c=1.237, *r=&c;
```







'P' is a Pointer of type unsigned integer that hold address of Signed integers.

'q' is a Pointer of type unsigned integer that hold address of Characters

'r' is a Pointer of type unsigned integer that hold address of float values.

Pointer size == Integer size

```
int a=7, *P;
char b='@', *q;
float c=1.2317, *r;
```

P=&b;	q=&a;	r=&a;
P=&c;	q=&c;	r=&b;

Invalid assignments.

type mismatch.

```
Void *V;  (or) V=&a;  (or) V=&c;
          V=&b;
```

Valid assignments

Topic : Pointers

Pointer Arithmetic : It defines the set of valid operations Performed on Pointers.

① Assign one Pointer to another Pointer of Same type.

```
int *p, *q;   p=q (or) q=p ✓
```

② Address assignment Using '&' operator.

```
int a=3;
int *p=&a;
```

Valid assignment.

```
int a=3, b=1000;
int *p;
p=b; (OR) p=1000;
```

Invalid assignment as they are treated as values but not addresses.

③ Assignment of NULL. `datatype *Pointer = NULL;`

④ Increment | Decrement.

```
p++; q--; ✓
```

⑤ Addition with Integer Constant.

```
p=p+3; // Valid
int a=3,*p;
p=p+a; // Invalid as 'a' is Variable
```

⑥ Subtraction with Integer Constant

⑦ Subtraction of 2 Pointers of Same type.

```
float *p, *q;
int x=p-q;
```

⑧ Comparision (==, !=)

if (p==NULL) (or) if (p!=NULL)

```
float k;
char j;
int i;
```

All operations are valid

```
i=7;
i=3.14;
i='#';
i++; i--;
i=i+5;
i=i-5;
i=i*3;
i=i/4;
i=i%10;
```

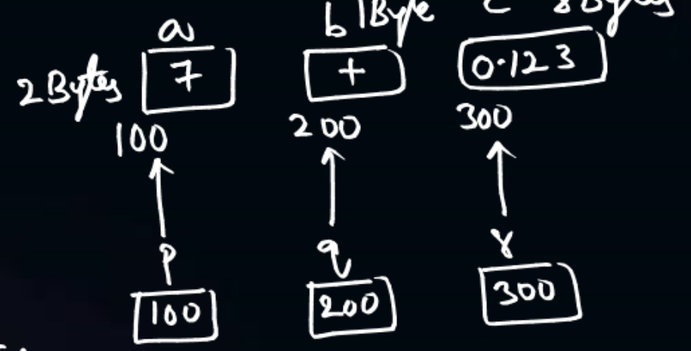
```
i==j
i==k
i!=k
i<k
i<<=3;
i>>=2;
if=1;
i!=20;
iff=j;
```

Topic : 1-D Array

Pointer Inc|Dec| Addition |Subtraction

Let 16-bit Processor

```
int a=7, *p;
char b='+', *q;
double c=0.123, *r;
p=&a; q=&b; r=&c;
```



p++; ⇒ p = p+(1*2) = 100+2 = 102

q++; ⇒ q = q+(1*1) = 200+1 = 201

r++; ⇒ r = r+(1*8) = 300+8 = 308

---

p = p+2 ⇒ 100+2*2 = 104

q = q-1 ⇒ 200-(1*1) = 199

r = r+3 ⇒ 300+(3*8) = 324

Let 'p' be a Pointer, Pointing to data of size 'N' Bytes.

$P \pm i == P \pm (i * N)$

NOTE:

The amount of Increment (or) addition and Decrement (or) Subtraction depends up on size of data to whom Pointer Points to.

Topic : 1-D Array

ARRAY : Group (or) Collection of Similar data items (or) Homo-geneous data items.

- 2 Types
  - One-dimensional Array (1-D array)
  - Multi-dimensional Array
    - 2-D array
    - 3-D array ....

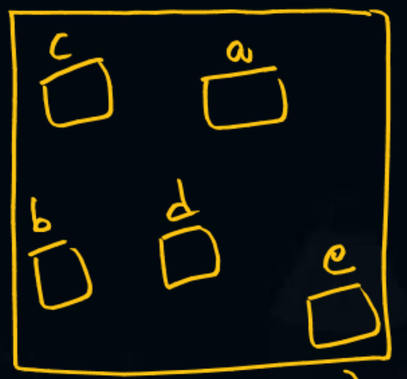
1-D Array : It is also known as Vector.

Syntax for Declaration: datatype Arrayname [Array size];

Ex:
```
int x[5];
float y[9];
char s[10];
struct s var[10];
int *p[10];
```

Let 32-bit Processor

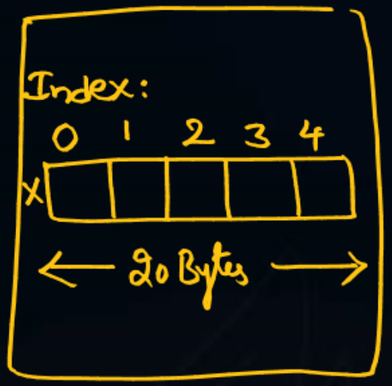
```
int a,b,c,d,e;
```



RAM(memory)

- Random memory allocation
- Accen takes more time
- Supports random accen

```
int x[5];
```



- Contiguous memory allocation
- Accen time will be Less.
- Also supports random access.

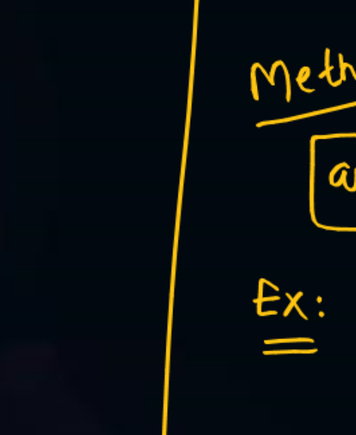
Topic : 1-D Array

* Starting address of any data == address

** Array is a Constant Pointer

Constant Pointer : A Pointer that holds fixed address.

```
int a = 3, b=5;
const * P= &a;
```




```
P= &b;
P=P+1;
P++;
P--;
```
} Invalid

```
*P = 7;
*P = *P + 5;
```
} valid

Value at address

Initialization of 1-D array

Method-1 : Along with declaration

```
datatype arrayname [size] = { value1, value2 --- };
```

Ex: `int X[5] = {10,20,30,40,50};`

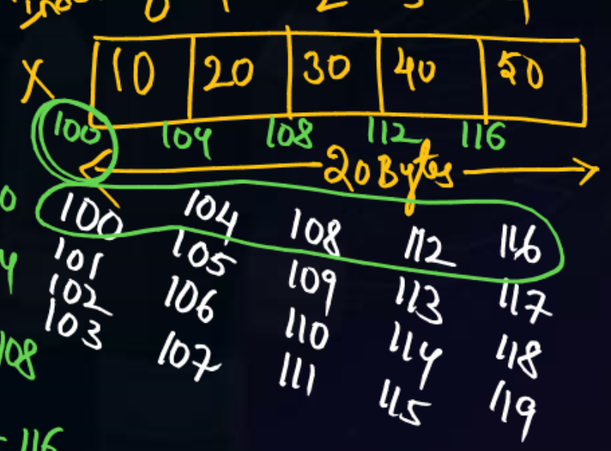
Method-2: Later declaration

```
array [index] = value;
```

*** Let 32-bit Processor

Start address of an array == Base address

Ex:
```
int X[5];   {X=100
X[0]=10;    {X[0]=100
X[1]=20;    {X[1]=104
X[2]=30;    {X[2]=108
X[4]=50;    {X[4]=116
```



Topic : 1-D Array

- Array name itself indicates Base address, as it is a constant Pointer.

```
int x[5] = {10,20,30,40,50};
```

	0	1	2	3	4
x	10	20	30	40	50
	100	104	108	112	116

x → 100

```
int y[5] = {11,22,33,44,55};
```

	0	1	2	3	4
y	11	22	33	44	55
	200	204	208	212	216

y → 200

value assignments

x[0] = y[0]

x[3] = y[2]

⋮

Valid

```
x = y; (or) y = x; // Invalid assignment.
```

```
// input and output 1-D array Elements
Void main()
{
  int x[5], i;
  Printf("Enter array values:");
  for(i=0; i<5; i++)
    Scanf("%d", &x[i]);
  // Print the array values
  for(i=0; i<5; i++)
    Printf("x[%d] value is %d", i, x[i]);
}
```

o/p: Enter array values:
10↵ 20↵ 30↵ 40↵ 50↵

x[0] value is 10
x[1] value is 20
x[4] value is 50

2 mins Summary

- Pointers — Null Pointer, Void Pointer, Constant Pointer
- Pointer datatype, size
- Pointer Arithmetic
- Arrays
  - 1-D array.

To be Contd ...

THANK - YOU